

Simulador *Cycle-Accurate* de uma Rede-em-Chip SPIN com Topologia Fat-Tree Quaternária em SystemC: Comutação *Wormhole*, Controle de Fluxo por Créditos e Arbitragem Probabilística

Breno Barbosa de Oliveira Felipe Marley de Oliveira Gomes
Fernando Gomes de Almeida Gustavo de Freitas Rodrigues
Curso de Ciência da Computação
Departamento de Informática e Matemática Aplicada (DIMAp)
Universidade Federal do Rio Grande do Norte (UFRN)
Trabalho Prático 2 — Redes-em-Chip (NoC)

Resumo—Este trabalho apresenta o projeto, a modelagem e a verificação de um simulador de ciclo exato (*cycle-accurate*) de uma Rede-em-Chip (*Network-on-Chip* — NoC) inspirada na arquitetura SPIN (*Scalable Programmable Integrated Network*), descrito em SystemC ao nível de transferência entre registradores (RTL). A rede adota uma topologia em Árvore Gorda (*Fat-Tree*) quaternária de dois níveis, com 4 roteadores-folha, 4 roteadores-raiz e 16 terminais de processamento. São implementados os três mecanismos fundamentais de uma NoC moderna: comutação *wormhole*, na qual mensagens são fragmentadas em *flits*; controle de fluxo baseado em créditos, que garante ausência de perda de pacotes por estouro de buffer; e um árbitro probabilístico híbrido, derivado das especificações da rede SPIN, que sorteia a cada ciclo a ordem de atendimento das portas para mitigar *starvation* e congestionamento. O roteamento combina uma fase ascendente adaptativa (escolha do enlace com maior número de créditos livres) com uma fase descendente determinística (endereçamento hierárquico pelo identificador do destino). Os resultados de simulação, obtidos pela injeção de uma mensagem *wormhole* de quatro *flits* entre dois terminais em sub-árvores distintas, demonstram a correta alocação e liberação de canais, a integridade do controle de fluxo e a remontagem completa do pacote no destino, validando funcionalmente o modelo proposto.

Index Terms—Rede-em-Chip, NoC, SPIN, Fat-Tree, SystemC, comutação *wormhole*, controle de fluxo por créditos, roteamento adaptativo, arbitragem probabilística.

I. INTRODUÇÃO

A evolução dos sistemas integrados em chip (*Systems-on-Chip* — SoC) levou à integração de dezenas a centenas de núcleos de processamento, memórias e aceleradores em um mesmo substrato de silício. Nesse cenário, os tradicionais barramentos compartilhados e as conexões ponto-a-ponto dedicadas tornaram-se gargalos: o barramento não escala em largura de banda nem em consumo, e as conexões dedicadas explodem em área de roteamento à medida que o número de núcleos cresce [2]. As Redes-em-Chip (NoCs) surgem como o paradigma de comunicação que substitui essas estruturas por uma infraestrutura de roteadores e enlaces comutados por

pacotes, à semelhança das redes de computadores, porém com restrições severas de área, latência e energia.

Entre as topologias propostas, a Árvore Gorda (*Fat-Tree*) destaca-se por oferecer largura de banda crescente em direção à raiz e múltiplos caminhos redundantes entre quaisquer dois nós, propriedades que favorecem o balanceamento de carga e a tolerância a congestionamento. A rede SPIN [1], uma das primeiras NoCs propostas na literatura, é justamente um exemplo canônico de NoC baseada em *fat-tree*, empregando roteamento adaptativo na subida e determinístico na descida.

Este trabalho tem por objetivo projetar, implementar e verificar um simulador *cycle-accurate* dessa arquitetura utilizando a biblioteca SystemC. Mais do que reproduzir a topologia, busca-se modelar fielmente, ao nível RTL, os mecanismos de hardware que governam o tráfego: a fragmentação de mensagens em *flits* (comutação *wormhole*), o controle de fluxo por créditos entre roteadores vizinhos e a política de arbitragem probabilística inspirada na SPIN. As contribuições do trabalho são: (i) um modelo modular e sintetizável-em-espírito da rede, com separação estrita entre interface e implementação; (ii) a implementação de uma máquina de estados *wormhole* com alocação e liberação explícita de canais por meio de uma tabela de travas; e (iii) a validação funcional do conjunto por meio de simulação dirigida por estímulos.

O restante do artigo organiza-se da seguinte forma. A Seção II apresenta o referencial teórico sobre NoCs, topologia *fat-tree*, comutação *wormhole*, controle de fluxo e a rede SPIN. A Seção III detalha o modelo implementado, seus componentes e o diagrama de blocos. A Seção IV discute as simulações realizadas e seus resultados. A Seção V conclui o trabalho.

II. REFERENCIAL TEÓRICO

A. Redes-em-Chip (NoC)

Uma NoC é composta por três elementos básicos: *roteadores*, responsáveis por encaminhar pacotes; *enlaces* (*links*), que conectam roteadores entre si e a núcleos; e *interfaces de*

rede (aqui denominadas terminais), que adaptam os núcleos ao protocolo da rede. O desempenho de uma NoC é caracterizado por três eixos de projeto fortemente acoplados: a *topologia* (o grafo de interconexão), o *algoritmo de roteamento* (a escolha do caminho) e o *mecanismo de controle de fluxo* (a gestão dos recursos de buffer e enlace) [3].

B. Topologia Fat-Tree Quaternária

A *fat-tree* é uma árvore na qual a capacidade dos enlaces aumenta em direção à raiz, eliminando o gargalo de banda típico de árvores convencionais. Em uma *fat-tree* quaternária (aridade 4), cada roteador possui quatro portas descendentes (em direção às folhas) e quatro ascendentes (em direção à raiz). A propriedade essencial é a existência de *múltiplos caminhos* ascendentes equivalentes entre uma folha e a raiz: qualquer roteador-raiz pode servir de ponto de inflexão para um pacote, o que habilita o roteamento adaptativo e o balanceamento de carga.

C. Comutação Wormhole

Na comutação *wormhole*, uma mensagem é dividida em unidades de controle de fluxo chamadas *flits* (*flow control units*). O primeiro, *flit-cabeçalho* (*header*), carrega o endereço de destino e, ao percorrer a rede, *aloca* (tranca) os canais de saída que utiliza, abrindo um “túnel”. Os *flits* de corpo (*payload*) seguem o cabeçalho em fila indiana sem necessidade de re-roteamento, e o *flit-cauda* (*tail*) libera os canais à medida que avança. A vantagem é a baixa latência e o reduzido requisito de *buffer* (basta espaço para alguns *flits*, não para o pacote inteiro); a desvantagem é que um pacote bloqueado pode reter canais ao longo de vários roteadores, exigindo cuidado com *deadlock* e *starvation* [3].

D. Controle de Fluxo por Créditos

O controle de fluxo *credit-based* previne o estouro dos *buffers* de entrada. Cada transmissor mantém um contador de *créditos* que reflete o número de posições livres no *buffer* FIFO do receptor a jusante. Um *flit* só é enviado se houver crédito disponível (*créditos* > 0); ao enviar, o transmissor decrementa o contador. Quando o receptor consome um *flit* de seu *buffer*, devolve um *crédito* ao transmissor, que incrementa o contador. Esse protocolo garante, por construção, a ausência de perda de *flits* por transbordo.

E. A Rede SPIN e a Arbitragem

A SPIN [1] é uma NoC *fat-tree* que populariza o esquema de roteamento em duas fases: *subida adaptativa*, em que o pacote pode tomar qualquer enlace ascendente livre (tipicamente o menos congestionado), e *descida determinística*, em que, a partir do ponto de inflexão, o caminho até o destino é único e calculado pelo endereço hierárquico. Quando múltiplas entradas disputam a mesma saída, um *árbitro* decide a ordem de atendimento. Políticas puramente fixas (prioridade estática) podem causar *starvation*; por isso, adota-se aqui uma arbitragem *probabilística*, que sorteia a cada ciclo a ordem de varredura das portas, garantindo que toda entrada seja eventualmente atendida.

Tabela I
ORGANIZAÇÃO DOS MÓDULOS DO SIMULADOR

Módulo	Responsabilidade
flit	Define o tipo Flit e o enum class FlitType (kHeader, kPayload, kTail).
terminal	Interface de rede: injeta a sequência <i>wormhole</i> e consome/remonta <i>flits</i> recebidos.
router	Roteador RSPIN: <i>buffers</i> , créditos, árbitro probabilístico e máquina <i>wormhole</i> .
fat_tree	Módulo <i>top-level</i> : instancia roteadores e terminais e realiza o cabeamento da malha.
trace	Registro de log opcional (ativável por variável de ambiente).
tests/	Conjunto de <i>testbenches</i> <i>sc_main</i> : cada um injeta um cenário e verifica a entrega.

III. MODELO IMPLEMENTADO

A. Visão Geral e Tecnologia

O simulador foi inteiramente descrito em **SystemC** (Accellera 3.0.2, padrão C++17), uma biblioteca de classes C++ para modelagem de hardware. A escolha do SystemC permite descrever o paralelismo intrínseco do hardware por meio de processos (SC_METHOD) sensíveis à borda de subida do relógio, canais (*sc_signal*) e portas (*sc_in/sc_out*), obtendo um modelo de *ciclo exato*. O relógio do sistema opera a um período de 10 ns (100 MHz), conforme definido no *testbench*.

O projeto adota separação estrita entre interface (*include/spin/*.hpp*) e implementação (*src/*.cpp*), com o código de simulação organizado como uma biblioteca reutilizável e os cenários de exercício isolados em *tests/*. A Tabela I resume os módulos.

B. Estrutura do Flit

A unidade de transporte é a struct Flit, que carrega o tipo do *flit*, os identificadores lógicos de origem (*src*) e destino (*dest*) em campos *sc_uint<10>*, uma carga útil textual (*data*) usada para inspeção da simulação, e um bit *valid* que sinaliza presença de dado no enlace naquele ciclo.

C. Topologia e Diagrama de Blocos

A rede contém 16 terminais ($T_0 \dots T_{15}$), 4 roteadores-folha ($L_0 \dots L_3$, nível 1) e 4 roteadores-raiz ($R_0 \dots R_3$, nível 2). Cada roteador-folha conecta-se a 4 terminais por suas portas descendentes e aos 4 roteadores-raiz por suas portas ascendentes, formando a malha *fat-tree* completamente conectada entre os dois níveis (Fig. 1). O terminal *i* pertence ao roteador-folha $\lfloor i/4 \rfloor$, na porta $i \bmod 4$.

D. O Roteador RSPIN

O coração do modelo é o módulo RSPIN_Router, parametrizado por identificador (*my_id*) e nível (*level*). Cada roteador possui quatro portas ascendentes e quatro descendentes, cada uma com seu par de sinais de dados e de crédito. Internamente mantém: *buffers* FIFO de entrada (*std::queue*) de profundidade 4 por porta; contadores de

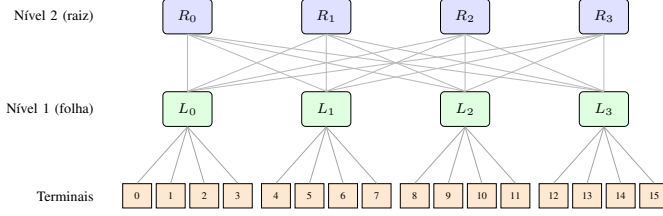


Figura 1. Topologia *fat-tree* quaternária: 4 roteadores-raiz, 4 roteadores-folha e 16 terminais. Todo enlace folha-raiz transporta dados em um sentido e créditos no sentido oposto.

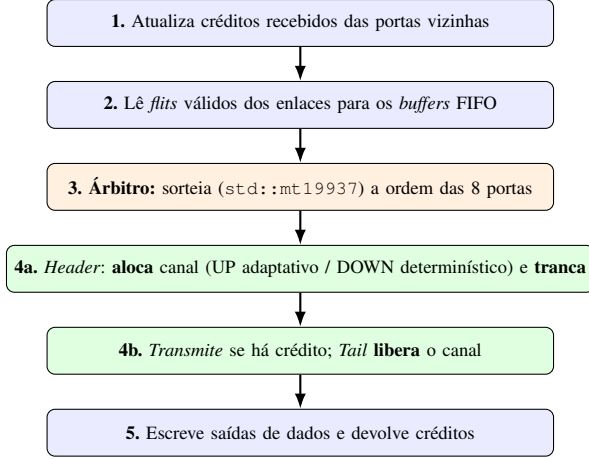


Figura 2. Pipeline lógico executado pelo processo `routing_logic` a cada ciclo de relógio (máquina *wormhole*).

Tabela II
ROLETA DE PRIORIDADES DO ÁRBITRO

Prob.	Ordem de varredura	Política
50%	45670123	Prioridade UP (descendo da raiz)
30%	01234567	Prioridade DOWN (subindo da folha)
15%	40516273	Misto, leve vantagem UP
5%	04152637	Misto, leve vantagem DOWN

crédito de saída; *flags* de ocupação de saída (`out*_busy`); e a tabela `input_locks[8]`, que registra, para cada uma das 8 entradas, qual direção e porta de saída ela “trancou”. O processo `routing_logic` é disparado a cada borda de relógio e executa o pipeline lógico da Fig. 2.

1) *Árbitro Probabilístico Híbrido*: A cada ciclo, o roteador sorteia um valor em $[0, 100)$ que define a ordem de varredura das 8 portas (índices 0–3 descendentes, 4–7 ascendentes), conforme as probabilidades da Tabela II, derivadas das especificações da SPIN. Essa aleatoriedade reordena dinamicamente as prioridades, evitando que uma entrada perca sistematicamente a disputa por uma saída (*anti-starvation*).

2) *Roteamento em Duas Fases*: Ao processar um *header*, o roteador calcula o roteiro a partir do destino: o roteador-folha de destino é $\lfloor \text{dest}/4 \rfloor$ e a porta local de terminal é $\text{dest} \bmod 4$.

- **Subida adaptativa**: se o roteador é de folha

(`level==1`) e o destino está em outra sub-árvore, o pacote deve subir. O roteador varre as 4 portas ascendentes livres e escolhe a de *maior número de créditos* disponíveis, isto é, o enlace menos congestionado.

- **Descida determinística**: caso contrário, a saída é unicamente determinada — na raiz, pela folha de destino; na folha, pela porta do terminal de destino.

Uma vez escolhida, a saída é registrada em `input_locks` e marcada como ocupada; somente o *flit-cauda* reverte essa trava, encerrando o “túnel” *wormhole*.

E. O Terminal

O módulo `Terminal` atua como interface de rede. Quando programado por `set_target()`, monta uma mensagem de 4 *flits* (1 *header* + 2 *payload* + 1 *tail*) e a injeta respeitando o crédito disponível. Em modo de recepção, lê o enlace de chegada, imprime o conteúdo de cada *flit* e, ao receber o *tail*, sinaliza a remontagem completa do pacote, devolvendo créditos ao roteador a montante.

IV. RESULTADOS

A. Metodologia e Cenário de Simulação

O modelo foi compilado com g++ 16.1 (C++17) e ligado à biblioteca **SystemC 3.0.2 da Accellera**; os logs e temporizações apresentados nesta seção foram *extraídos de execuções concretas*, e não de estimativas analíticas. Cada cenário de simulação é um *testbench* `sc_main` (diretório `tests/`) que constrói a rede, aplica o relógio e o *reset*, injeta o tráfego e executa a simulação de ciclo exato — ou seja, *os testes são a própria simulação*, acrescidos de uma verificação automática da entrega. No cenário básico (`test_single_flow`): aplica-se *reset* por 20 ns, libera-se a rede e, em $t = 30$ ns, configura-se o tráfego `configure_traffic(1, 14)` — o terminal T_1 envia uma mensagem *wormhole* de 4 *flits* ao terminal T_{14} ; a simulação prossegue por 300 ns. Esse cenário é deliberadamente um *caminho longo*: origem e destino residem em sub-árvores distintas, exigindo subida até a raiz e descida até a folha oposta. O log detalhado é emitido sob demanda (variável de ambiente `SPIN_TRACE`).

B. Caminho Esperado

Aplicando o endereçamento hierárquico ao destino 14: folha de destino = $\lfloor 14/4 \rfloor = 3$ e porta de terminal = $14 \bmod 4 = 2$. A origem T_1 pertence à folha $\lfloor 1/4 \rfloor = 0$. Como as sub-árvores diferem ($0 \neq 3$), o pacote percorre:

$$T_1 \rightarrow L_0 \xrightarrow{\text{UP adapt.}} R_0 \xrightarrow{\text{DOWN det.}} L_3 \xrightarrow{\text{DOWN det.}} T_{14}.$$

Em L_0 , todas as portas ascendentes iniciam com 4 créditos; o roteamento adaptativo seleciona a primeira de maior crédito, a porta UP 0, conectada a R_0 . Em R_0 (nível 2), a descida determinística aponta para a folha 3 (porta DOWN 3); em L_3 , a descida final aponta para a porta DOWN 2, que serve T_{14} .

Tabela III
OCUPAÇÃO DOS ENLACES POR CICLO (EXECUÇÃO REAL)

t (ns)	$T_1 \rightarrow L_0$	$L_0 \rightarrow R_0$	$R_0 \rightarrow L_3$	$L_3 \rightarrow T_{14}$
40	PAY#1	HEADER	—	—
50	PAY#2	PAY#1	HEADER	—
60	TAIL	PAY#2	PAY#1	HEADER
70	—	TAIL	PAY#2	PAY#1
80	—	—	TAIL	PAY#2
90	—	—	—	TAIL

C. Simulação 1: Alocação do Canal pelo Header

O *flit-cabeçalho* dispara a fase de alocação em cada salto. O excerto de log a seguir, *copiado da saída de console da execução*, evidencia a intervenção do árbitro e o trancamento adaptativo do enlace ascendente em L_0 :

```
[ PACOTE INICIADO ] @ 30 ns | Origem: 1 Destino: 14
[ ARBITRO ] Roteador Nivel 1 ID 0 | caiu em 50% (Prioridade UP) →Atendendo Entrada 1
→[HW] Roteamento Adaptativo trancou porta UP 0
»@ 40 ns | Roteador Nivel 1 (ID 0) despachou [HEADER] →Porta UP 0
[ ARBITRO ] Roteador Nivel 2 ID 0 | caiu em 30% (Prioridade DOWN) →Atendendo Entrada 0
»@ 50 ns | Roteador Nivel 2 (ID 0) despachou [HEADER] →Porta DOWN 3
[ ARBITRO ] Roteador Nivel 1 ID 3 | caiu em 30% (Prioridade DOWN) →Atendendo Entrada 4
»@ 60 ns | Roteador Nivel 1 (ID 3) despachou [HEADER] →Porta DOWN 2
«< @ 70 ns | Terminal 14 RECEBEU [HEADER] | Dado: »INICIO_DA_MENSAGEM «
```

A leitura confirma três pontos: (i) a cada salto o árbitro registra em qual faixa da roleta caiu (neste *run*: L_0 em 50% e os demais em 30%) e qual entrada atendeu; (ii) em L_0 o roteamento adaptativo de fato trancou a porta UP 0 (a primeira de maior crédito); e (iii) o cabeçalho avança um salto por ciclo de 10 ns (40 ns→50 ns→60 ns→70 ns), caracterizando o avanço *pipeline* típico da comutação *wormhole*.

D. Simulação 2: Pipeline do Corpo e Remontagem

Após a abertura do túnel, os *flits* de corpo e cauda seguem o mesmo caminho sem nova alocação (a trava já existe), formando um *pipeline sistólico*: em um mesmo ciclo, saltos distintos transportam *flits* distintos da mesma mensagem. A Tabela III reconstrói, a partir do log de execução, a ocupação de cada enlace ao longo do tempo.

No destino, o terminal remonta a mensagem completa, conforme o log real:

```
«< @ 80 ns | Terminal 14 RECEBEU [PAYLOAD] | Dado: Ola mundo :) #1
«< @ 90 ns | Terminal 14 RECEBEU [PAYLOAD] | Dado: Ola mundo :) #2
«< @ 100 ns | Terminal 14 RECEBEU [TAIL] | Dado: »FIM_DA_MENSAGEM «
→(Pacote completamente recebido e remontado!)
```

A ordem de chegada (HEADER, PAYLOAD#1, PAYLOAD#2, TAIL) é idêntica à ordem de injeção, comprovando a preservação de ordem do *wormhole* e a integridade da mensagem. A latência fim-a-fim do cabeçalho é de 4 ciclos (40 ns, de $t = 30$ ns a $t = 70$ ns) e o pacote inteiro é entregue em 100 ns.

E. Verificação e Depuração

Durante a verificação por execução, observou-se inicialmente que o terminal de destino recebia o conteúdo do primeiro *payload* duplicado, sem jamais exibir o segundo. A investigação revelou a causa-raiz no operador de igualdade

Tabela IV
BATERIA DE CASOS DE TESTE (TODOS APROVADOS)

Cenário	Fluxos	Propriedade verificada
Fluxo único	$T_1 \rightarrow T_{14}$	Caminho completo (subida adaptativa + descida determinística).
Fluxo local	$T_0 \rightarrow T_3$	Roteamento local na mesma folha, sem subir à raiz.
Convergência	$T_0, T_8 \rightarrow T_7$	Ausência de perda de <i>flits</i> sob contenção de saída.
Permutação	8 fluxos disjuntos	N fluxos concorrentes sem <i>deadlock</i> nem perda.
Muitos-para-um	$T_0 \dots T_3 \rightarrow T_{10}$	Serialização justa de 4 fluxos para um destino.

da estrutura Flit: por não comparar o campo *data*, dois *payloads* de mesmo tipo, origem e destino eram considerados idênticos pelo canal *sc_signal*, que então suprimia a atualização do valor (semântica padrão do SystemC, que só propaga quando novo != atual). A correção consistiu em incluir o campo *data* na comparação, após o que a simulação passou a entregar corretamente #1 e #2, como mostra o log acima. O episódio ilustra o valor da simulação *cycle-accurate* para expor defeitos sutis de modelagem que a inspeção estática do código não revela.

Um segundo defeito foi exposto sob tráfego concorrente: quando dois fluxos convergiam para a mesma porta de saída, o *flit-cauda* de um deles, ao liberar o canal, permitia que o outro fluxo o readquirisse *no mesmo ciclo*, sobrescrevendo o sinal de saída e perdendo a cauda. A correção restringe a no máximo uma escrita por porta de saída por ciclo, preservando a integridade das mensagens sob contenção.

F. Bateria de Casos de Teste

Para assegurar que o modelo se comporta conforme especificado, foi desenvolvida uma bateria de casos de teste autoverificáveis, integrados ao CTest. Cada teste injeta um cenário, executa a simulação e *compara automaticamente* o número de pacotes remontados em cada destino com o valor esperado, retornando falha em caso de divergência. A Tabela IV resume os cinco cenários; **todos passam**.

Dois cenários merecem destaque. No teste de *convergência*, os dois pacotes destinados a T_7 são entregues de forma serializada, com as caudas chegando em 100 ns e 140 ns (intervalo de 40 ns, exatamente uma mensagem de quatro *flits*), confirmando a correção do defeito de contenção. No teste *muitos-para-um*, os quatro pacotes endereçados a T_{10} chegam em 100 ns, 140 ns, 180 ns e 220 ns — espaçamento regular que evidencia a serialização justa e a ausência de perda mesmo sob forte disputa por um único recurso de saída.

G. Análise de Formas de Onda

A Fig. 3 ilustra a temporização dos sinais ao longo do caminho. Em cada salto, o sinal *valid* do enlace sobe por um ciclo quando um *flit* é despachado, e o sinal de *credit*

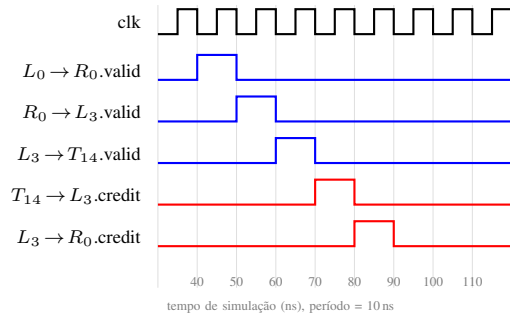


Figura 3. Formas de onda do *flit-header* ao longo do caminho, com os tempos da execução real. Os pulsos *valid* (azul) propagam-se um ciclo por salto ($L_0 \rightarrow R_0$ em 40 ns, $R_0 \rightarrow L_3$ em 50 ns, $L_3 \rightarrow T_{14}$ em 60 ns); os pulsos de *credit* de retorno (vermelho) confirmam o consumo e a liberação de *buffer* a jusante.

de retorno sobe um ciclo depois, quando o *flit* é consumido a jusante, devolvendo a posição de *buffer*. O deslocamento progressivo de um ciclo entre os enlaces $L_0 \rightarrow R_0 \rightarrow L_3 \rightarrow T_{14}$ evidencia o avanço *wormhole*.

H. Discussão

Os resultados validam funcionalmente os três mecanismos centrais. A **comutação wormhole** é confirmada pela alocação no *header*, pelo fluxo em fila dos *payloads* e pela liberação no *tail*. O **controle de fluxo por créditos** é confirmado pela ausência de perda de *flits* (todos os 4 chegam) e pelo retorno de créditos visível nas formas de onda. O **roteamento adaptativo** escolheu o enlace ascendente menos congestionado, e a **descida determinística** entregou o pacote exatamente ao T_{14} . Por fim, o **árbitro probabilístico** interveio em cada salto, registrando faixas distintas da roleta, o que ilustra a reordenação dinâmica de prioridades projetada para mitigar *starvation*.

V. CONCLUSÕES

Este trabalho apresentou um simulador *cycle-accurate* em SystemC de uma Rede-em-Chip SPIN com topologia *fat-tree* quaternária de dois níveis, contemplando 8 roteadores e 16 terminais. O modelo implementa, de forma integrada e modular, os mecanismos que definem o desempenho de uma NoC moderna: comutação *wormhole* com alocação e liberação explícita de canais por meio de uma tabela de travas; controle de fluxo *credit-based* que assegura, por construção, a ausência de perda de *flits*; roteamento híbrido (subida adaptativa por créditos e descida determinística por endereço hierárquico); e um árbitro probabilístico inspirado na SPIN para mitigar *starvation*.

A verificação funcional, conduzida por um cenário de tráfego de caminho longo ($T_1 \rightarrow T_{14}$), demonstrou a correta abertura do túnel *wormhole*, o avanço *pipeline* de um salto por ciclo, a preservação da ordem dos *flits* e a remontagem completa da mensagem no destino, com os créditos retornando conforme esperado. Os logs de simulação e as formas de onda corroboram, ponto a ponto, o comportamento projetado.

REFERÊNCIAS

- [1] P. Guerrier and A. Greiner, "A generic architecture for on-chip packet-switched interconnections," in *Proc. Design, Automation and Test in Europe (DATE)*, 2000, pp. 250–256.
- [2] L. Benini and G. De Micheli, "Networks on chips: a new SoC paradigm," *IEEE Computer*, vol. 35, no. 1, pp. 70–78, Jan. 2002.
- [3] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA: Morgan Kaufmann, 2004.
- [4] IEEE, *IEEE Standard for Standard SystemC Language Reference Manual*, IEEE Std 1666-2011, 2012.
- [5] T. Bjerregaard and S. Mahadevan, "A survey of research and practices of Network-on-Chip," *ACM Computing Surveys*, vol. 38, no. 1, pp. 1–51, 2006.